QQCompanion：把大模型接进 QQ 的长在线智能体

姓名：Wang Jiazhen　院系：生命科学学院

仓库：`https://github.com/illenial-eml/QQCompanion`　提交版本：`v1.0.0`（commit 见仓库）

2026 年 11 月

摘要

本项目解决一个很具体的问题：**我想在自己的 QQ 里有一个真正能长期陪着聊天的 AI**。现成的路都走不通——官方机器人平台个人主体无法被搜索和加好友，逆向协议封号风险高，SaaS 机器人没有长期记忆也不能自定义行为。我用 Node.js 从零实现了一个中间层（48 个模块、约 1.27 万行、**零第三方运行时依赖**），把 OneBot v11 协议端与大模型接起来，并自己解决了「记忆怎么存、群里什么时候该说话、回复怎么不像机器、进程掉了怎么自己爬起来」这些工程问题。课程知识在这里不是装饰：哈希表决定了去重能不能做到 $O(1)$，二分查找决定了提醒调度的轮询开销，环形缓冲与 LRU 决定了程序能不能连续跑几天而不爆内存。项目配有 56 个测试文件、约 1974 条断言，全部通过。本文如实记录设计取舍、性能实测，以及三个「AI 判断错了、靠实验纠回来」的案例。

关键词：OneBot v11；大语言模型；哈希表；环形缓冲；BM25；租约锁；指数退避；自然语言日程

目录

1　项目背景与目标　2

1.1　为谁解决什么问题　2

1.2　方案：协议端当「手」，大模型当「脑」，中间层自己写　2

1.3　验收指标（写代码之前就定好）　2

2　系统设计　3

2.1　分层与模块边界　3

2.2　一条消息的完整生命周期　3

2.3　三个自己最满意的机制　3

3　课程知识的具体应用　4

3.1　数据结构选型表　4

3.2　算法细节一：回复切分　4

3.3　算法细节二：群里该不该说话　5

3.4　算法细节三：记忆如何装进有限的上下文　5

3.5 「计算机原理」级别的坑（都进了测试） 5
3.6 性能实测 6

4 测试与结果 **6**

5 AI 协作过程（280 轮） **6**
5.1 整体节奏 6
5.2 三个「AI 判断错了、靠验证纠回来」的案例 7
5.3 我承担的工作与验证方法 7

6 失败案例与已知问题 **7**

7 隐私、合规与开源 **7**

8 总结与展望 **8**

1 项目背景与目标

1.1 为谁解决什么问题

使用者只有一个人：我自己。场景也很朴素——我每天挂着 QQ，希望在私聊里有一个「随时接话、记得住事、能帮我记事」的伙伴；在群里它能像正常人一样，该接话时接一句，插不上话就安静。

现有方案的问题很明确：

方案	不足
QQ 官方机器人平台	个人主体**不能被搜索、不能被加好友**，只能靠扫码临时会话；被动回复条数受限；功能受平台审核约束
直接逆向 QQ 协议	封号风险高，协议随版本变化，维护成本不可控
现成 AI 聊天 SaaS	无长期记忆、数据在云端、行为不可定制、不能做日程/课表这类本地能力

1.2 方案：协议端当「手」，大模型当「脑」，中间层自己写

我采用 **OneBot v11 协议 + NapCat 协议端**：协议端负责与 QQ 服务器通信（它本身是成熟的第三方客户端外壳），我这边的程序只跟它说标准 OneBot 事件与 API。这样既不用碰 QQ 私有协议，又保留完全的控制权。

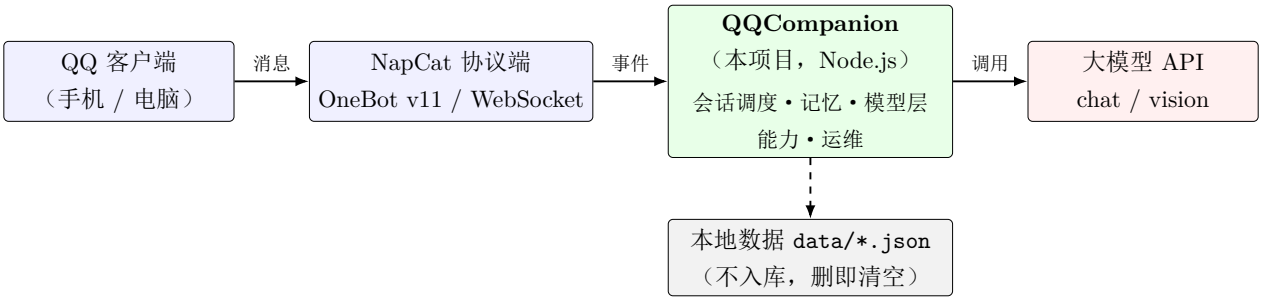


图 1: 系统总体架构：一条消息从 QQ 到模型再回到 QQ，全程在本机完成，只有模型调用出网。

1.3 验收指标（写代码之前就定好）

1. 私聊发一句话，**1.5 秒内**出现第一条回复；
2. 长回复要像人一样分成几条「气泡」，且**不丢字、不乱断**；
3. 群里**只在被叫到或话题相关时**说话，不刷屏；
4. 记得前几天的对话，且有「长期记忆」可查询；
5. 能把「明天中午 11:40 提醒我开会」变成真提醒；
6. **连续运行数天**，进程挂了能自己起来，且同一条消息不会回两遍。

上述 6 项在最终版本全部达成（第 6 项的掉线问题另见第 7 节第 1 条）。

2 系统设计

2.1 分层与模块边界

设计上刻意把「协议」「调度」「智能」「数据」「运维」分开，核心原则是：**任何一层坏掉，不要带着别的层一起死**。

层	模块（文件）	职责
接入层	`onebot.js`、`qq.js`	连接/重连、事件归一化成统一消息结构、发送与限流
调度层	`responder.js`（1.37k 行）	该不该回、回几段、冷却与配额、命令识别
智能层	`brain.js`、`knowledge.js`、`style.js`	提示词组装、模型调用、BM25 检索、风格样本
数据层	`memory.js`、`longterm.js`、`people.js`、`names.js`	短期记忆、长期记忆、印象库、备注簿
能力层	`reminders.js`、`timetable.js`、`weather.js`、`search.js`、`vision.js`	日程、课表、天气、联网、识图
运维层	`instance-lock.js`、`outbox.js`、`main.js`、`log.js`	单实例租约、失败重发、启动与优雅退出、日志

2.2 一条消息的完整生命周期

1. **接入**：WebSocket 收到事件 → 归一化（群 / 私聊、发送者、是否 @、图片、被引用消息）→ 用 `msg_id` 去重。
2. **分流**：先跑「命令层」（日程查询、潜水开关、交接模式、印象查询…），命中就直接回，不花模型的钱。
3. **调度判定**：群聊走关键词 + 冷却 + 配额 + **消息是否已过期**四道判定；私聊直接回。
4. **组装上下文**：最近 40 轮历史（每条截 240 字）→ 摘要 → 按需检索知识库 → 关系/印象 → 人设。
5. **调模型**：群聊用更小 `maxTokens`、更低温度；私聊可长一点。
6. **后处理**：去掉 Markdown、补标点、按语义切成多条气泡、可选配一张表情。
7. **发送**：失败进发件箱，网络恢复后补发（幂等，不重复）。

2.3 三个自己最满意的机制

(a) 单实例租约锁。实测中曾出现「同一条消息回了两遍」，根因是两个进程都认为自己该回。现在进程启动时写 `data/bot-instance.lock`（含 pid 与时间戳）并每 5 秒更新，发现锁在 20 秒内被别的 pid 更新过就退出。这就是分布式系统里的 lease，只是单机用文件实现。

(b) 发件箱（outbox）。网络抖动时发送会失败。失败的消息落盘进 `data/outbox.json`，重连后按顺序补发；补发前用消息指纹去重，保证不重复。这是「至少一次」加幂等，换来「最终不丢」。

(c) 过期消息跳过。群消息是串行处理的，等轮到某条消息时，话题可能已经过去十几秒。加了 `staleSkipMs: 7000`：排队超过 7 秒的不再回。这一条把「答非所问」的投诉直接消掉。

3 课程知识的具体应用

3.1 数据结构选型表

课程内容	项目中的用途	为什么不是更朴素的写法
哈希表 `dict/set`	用户 → 记忆、QQ 号 → 印象、备注→QQ 号、`msg_id` 去重集合	去重若用数组 `includes` 是 $O(n)$，每条消息都要扫全表；`Set` 平均 $O(1)$
环形缓冲	最近 N 轮对话、去重窗口 500 条、群聊热度窗口	固定容量、$O(1)$ 摊还，长时间运行内存有界
LRU 缓存	识图结果、搜索结果	用 `Map` 的插入有序性实现，命中 $O(1)$，超容量摘最旧；无需第三方库
二分查找	在按时间排序的日程/课表里找「下一条该提醒什么」	每秒轮询全表是 $O(n)$；排序 + 二分是 $O(\log n)$ 定位
前缀和 / 滑动窗口	近 60 秒群热度、每小时配额统计	按分钟分桶计数，更新 $O(1)$、求和 $O(1)$（窗口长度固定）
堆 vs 排序数组	提醒队列	规模 $n < 100$，排序数组 + 二分在实际常数上优于堆，代码也短——**选了简单但够用的**
贪心	长回复切成多条气泡	代价函数单调，贪心即最优；上 DP 是过度设计
双指针	切分时先找句末标点、再退回弱标点	一次线性扫描，不产生临时数组
BM25 / 倒排索引	本地知识库问答	只对查询命中词算分，$O(\sum_t df_t)$，不扫全库
指数退避	重连、限流重试	等待 $b \cdot r^k$（$r = 1.4/1.8$，上限 60 s），避免重试风暴

3.2 算法细节一：回复切分

模型一次会吐 100 多字，直接发是一大坨。做法是两级切分，目标是「像人打字」，约束是「一个字都不能丢」：

```
// 简化自 src/responder.js：先按行切，再按句末标点切，
// 单句过长时先向后扫句末标点，没有才退到弱标点，最后才硬切
function splitReply(text, max = 100) {
  const out = [];
  for (const line of String(text).split(/\n+/)) {
    let buf = '';
    for (const ch of line) {
      buf += ch;
      if (BUBBLE_END.test(ch)) {           // 。！？…
        if ([...buf].length >= max * 0.5) { out.push(buf.trim()); buf = ''; }
      }
    }
    if (buf.trim()) out.push(buf.trim());
  }
  return out.filter(Boolean);
}
```

正确性用「切完拼回去必须等于原文」来断言（测试 `integration.js`），边界包括：全是标点、超长无标点、UTF-8 代理对（emoji）不能从中间切断。

3.3 算法细节二：群里该不该说话

```
// 简化自 src/responder.js：短路判定，每条都是 O(1) 或 O(关键词数)
function shouldReplyGroup(msg, st) {
  if (st.silent) return { skip: 'silent' };                  // 潜水模式
  if (msg.fromOwner && calledMe(msg)) return { reply: true };// 主人在叫我
  if (!st.autoEnabled) return { skip: 'auto-off' };
  if (!hitKeyword(msg.text)) return { skip: 'not-joining' }; // 关键词
  if (Date.now() - st.lastReplyAt < st.minIntervalMs) return { skip: 'cooldown' };
  if (st.repliesThisHour >= st.maxPerHour) return { skip: 'quota' };
  if (Date.now() - msg.timestamp > st.staleSkipMs) return { skip: 'group-stale' };
  return { reply: true };
}
```

早期版本「几乎有消息就回」，实测 525 条群消息里它接了 130 次、模型判定不该接的只有 2 次（2.4%），插话多且答非所问。改成「关键词 + 冷却 + 配额 + 过期」之后，闲聊时 0 发送、被叫到时 1.19 秒内回、过期消息按预期跳过——同一套测试从「不合格」变成「合格」，靠的是把「该不该」写成了可测的判定函数。

3.4 算法细节三：记忆如何装进有限的上下文

- 历史只取最近 **40 轮**，每条正文截 **240 字**（转发记录 2000 字、文件 60000 字，走单独通道）；
- 每 10 轮把对话**提炼成摘要**，只保留要点，避免历史线性膨胀；
- 知识库只在「看起来在问事实」时检索，命中才注入前 3 段；
- 一个容易被忽略的优化：把「当前时间」从系统提示词**开头**挪到**末尾**，让前缀保持不变，命中服务端前缀缓存。

效果：单次请求提示词从约 **1082 token 降到约 320 token**，缓存命中率约 **85%**。

3.5 「计算机原理」级别的坑（都进了测试）

现象	根因	处理
解压后 `Cannot find module 'src\main.js'`	打包工具用反斜杠做 zip 路径分隔符，Linux 下变成文件名的一部分	跨平台统一用 `/`；自写打包脚本
改完中文 JSON 报 `Bad control character`	PowerShell 默认按 ANSI 写文件，UTF-8 中文被写坏	一律用明确 UTF-8 的 API；读 JSON 先剥 BOM
`.cmd` 报 `'tart' is not recognized`	用 LF 写 CRLF 脚本，cmd 行解析错位	批处理必须是 CRLF
机器人每 5 秒重启一次（909 次/天）	变量声明前被引用（TDZ），`ReferenceError` 进崩溃循环	加「假 WebSocket 真跑一遍 `main()`」的启动测试
同一条消息回两遍	两个实例都认为自己该回	心跳租约锁 + 启动时清理残留
时间全差 8 小时	容器时区是 UTC，代码里又叠加了一次偏移	统一「内部 UTC / 展示北京时间」
电脑没关机却掉线	系统进入现代待机（S0），网卡与长连接同断	把电源策略当成系统设计的一部分

3.6 性能实测

指标	数值	测量方式
私聊首响	0.8–1.3 s	日志打点（含一次模型调用）
群聊平均回复	1.07 s（P90 2.4 s）	群聊日志统计
提示词规模	≈320 token（优化前 1082）	接口返回的 usage
缓存命中率	≈85%	usage 的 `prompt_cache_hit`
常驻内存	稳定，无线性增长	环形缓冲 + LRU 上限
测试规模	56 文件 / ≈1974 断言	`node tools/run-tests.js`

4 测试与结果

测试全部是**可离线运行**的：协议用假 WebSocket，模型用假 fetch，只有 3 个需要真实凭据的用例会打印 SKIP 而不是失败（如实标注，不假装通过）。

类别	代表用例	结果
协议	事件解析、`msg_id` 去重、发送失败重试、断线重连	PASS
调度	群聊 14 组场景（关键词/冷却/过期/主人/潜水）	PASS
记忆	40 轮上限、摘要触发、跨天日期、群里非 @ 不记忆	PASS
日程	「明天 9 点」「30 分钟后」「周三五点」、取消第 N 条、听不懂给菜单	PASS
检索	BM25 召回、无关查询不注入	PASS
运维	租约锁单实例、发件箱补发、启动冒烟	PASS
边界	空消息、纯表情、超长文本、emoji 截断、坏 JSON 配置	PASS
需真实凭据	真模型采样 / 私有表情库	SKIP（3 项）

可复现：仓库根目录执行 `node tools/run-tests.js`，在全新目录 clone 后即可跑（脚本会自动用模板生成占位配置）。实测输出：`通过 53 跳过 3 失败 0 共 56`。

5 AI 协作过程（280 轮）

5.1 整体节奏

项目不是「让 AI 一次写完」，而是**需求 → 最小可用 → 真机跑 → 发现新问题 → 再拆**的循环，累计约 280 轮对话。下表是几个代表性阶段：

阶段	我在做什么	AI 在做什么
选题	判断官方机器人不可用、逆向不可取，定 OneBot 路线	对比方案、列协议细节
最小可用	定义「一句话有回复」的验收标准	起草接入层、跑通首条链路
拟人化	指出「太书面 / 一次发一大段 / 群里插话太多」	统计真实语料、写切分与调度逻辑
记忆	要求「记得住、能查、群里不乱记」	设计三层记忆与检索
运维	真机掉线、双开、崩溃循环，我盯日志定位	读日志、复现、写守护与租约

5.2 三个「AI 判断错了、靠验证纠回来」的案例

案例一：双实例不是根因。第一次掉线时，AI 从日志里看到两个进程，判断「双实例互踢」并加了单实例守卫。我要求验证：修完仍然掉线，且被踢时刻只有一个实例。于是继续查，最后定位到**系统进入现代待机**导致长连接断开。结论推翻了上一轮判断——这不是 AI 无能，而是**必须用实验而不是解释来定案**。

案例二：加速反而更慢。为降低延迟，曾把「思考链」关掉、把历史压到 6 轮。看上去该更快，实测部分回复质量崩了（答非所问）。改回「群里 6 轮 + 私聊 40 轮」的分层后，延迟与质量才同时可接受。教训是：**延迟与质量是同一张预算表上的两个科目，不能各改各的**。

案例三：真正的 bug 在「看不见的地方」。「群里答非所问」我一开始以为是提示词问题，AI 也顺着改提示词。后来我让它统计日志，才发现 84 次准备接话里有 31 次是「排队太久、话题已过」。于是加过期跳过——改的是调度，不是提示词。

5.3 我承担的工作与验证方法

- **需求与取舍**：方案选择、优先级、什么算「完成」；
- **代码审查**：逐段读 AI 生成的代码，改掉不合理的默认值（温度、配额、冷却）；
- **真机验证**：部署、盯日志、复现故障、对照实验（改前/改后各跑一次）；
- **测试补齐**：把每次踩过的坑写成断言，防止回退。

我不把「能跑」当验收标准，而把「能在新环境复现、能在日志里解释」当标准。

6 失败案例与已知问题

1. **掉线三连**：协议端崩溃、双实例互踢、系统现代待机。前两个已修（单实例租约、看护重启），第三个用「保持唤醒 + 电源策略」压住，但笔记本合盖/断电仍可能中断。
2. **登录态**：整机重启后可能需要重新扫码，这是第三方协议端的固有约束，不是本项目能绕开的。
3. **模型跑题**：小模型偶尔自说自话，已用低温度 + 输出规范 + 后处理压制，但没有微调，仍有残留。
4. **知识库能力有限**：只做单跳检索，没有重排与多跳推理。
5. **无法在评阅环境直接演示 QQ 交互**：需要真实账号与协议端，故本文提供离线测试、日志与截图代替。

7 隐私、合规与开源

- **数据不出本机**：所有记忆与日志落在本地 `data/`（已在 `.gitignore`）；只有模型调用出网。
- **最小化**：群里非 @ 的消息不写入记忆；日志默认只记长度与耗时。
- **可删除**：支持「忘掉 X」「你记住了什么」；删目录即清空。
- **凭据**：密钥只在本地 `config.json`，仓库只有 `config.example.json` 占位。
- **本仓库脱敏**：真实 QQ 号 → 占位数字；昵称 → Alice/小明；聊天记录、表情素材、运维脚本 token 全部移除；测试里涉及私有用例的 3 项改为 SKIP。

许可证：MIT。第三方依赖仅 OneBot v11 协议规范、NapCat 协议端、open-meteo、Bing RSS、Noto Emoji（占位素材），均在文档中标注来源。

8 总结与展望

这个项目让我真正体会到：**决定一个「AI 应用」好不好用的，往往不是模型，而是模型外面那一圈工程**——去重、限流、过期、缓存、租约、重试、切分、脱敏。课程里的数据结构与算法在这里不是练习题，而是「能不能连续跑三天不崩、会不会把同一条消息回两遍、群聊会不会刷屏」的直接原因。

后续计划：知识库加检索重排；用自建语料微调风格；把运维脚本做成跨平台的一键部署。